

API Testing Automation Report

NAME:Karthikeyan S

Date:29-06-2026

Title

API Automation Testing using Rest Assured Framework

1. Introduction

The purpose of this testing is to validate the functionality of a REST API endpoint and ensure that the API returns the expected response data correctly.

The API used for testing is:

Base URL:

<https://jsonplaceholder.typicode.com>

Endpoint:

GET /posts/1

The API returns post details in JSON format.

2. Objective

As a QA Engineer, the objective is to:

- Send a GET request to retrieve a post
 - Validate the API response
 - Verify response status code
 - Validate JSON response fields
 - Ensure mandatory fields are present
 - Implement assertions using Rest Assured
-

3. Technologies Used

Technology Purpose

Java 17 Programming Language

Maven Dependency Management

4. Maven Dependencies Used

The following dependencies were added in pom.xml:

- Rest Assured
- JUnit Jupiter

These dependencies help in API request handling, response validation, assertions, and test execution.

```
<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>rest-assured</artifactId>
  <version>5.4.0</version>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>${junit.version}</version>
  <scope>test</scope>
</dependency>
```

5. Framework Design

The framework follows a reusable and scalable structure.

Framework Components

BaseTest

Responsible for:

- Setting Base URI
- Common test initialization

```
@BeforeAll
public static void setup() {

    RestAssured.baseURI = "https://jsonplaceholder.typicode.com";

}
```

SpecFactory

Responsible for:

- Request Specification
- Response Specification
- Reusable configurations

Test Class

Responsible for:

- Executing API requests
- Validating response data
- Assertions

6. Project Folder Structure

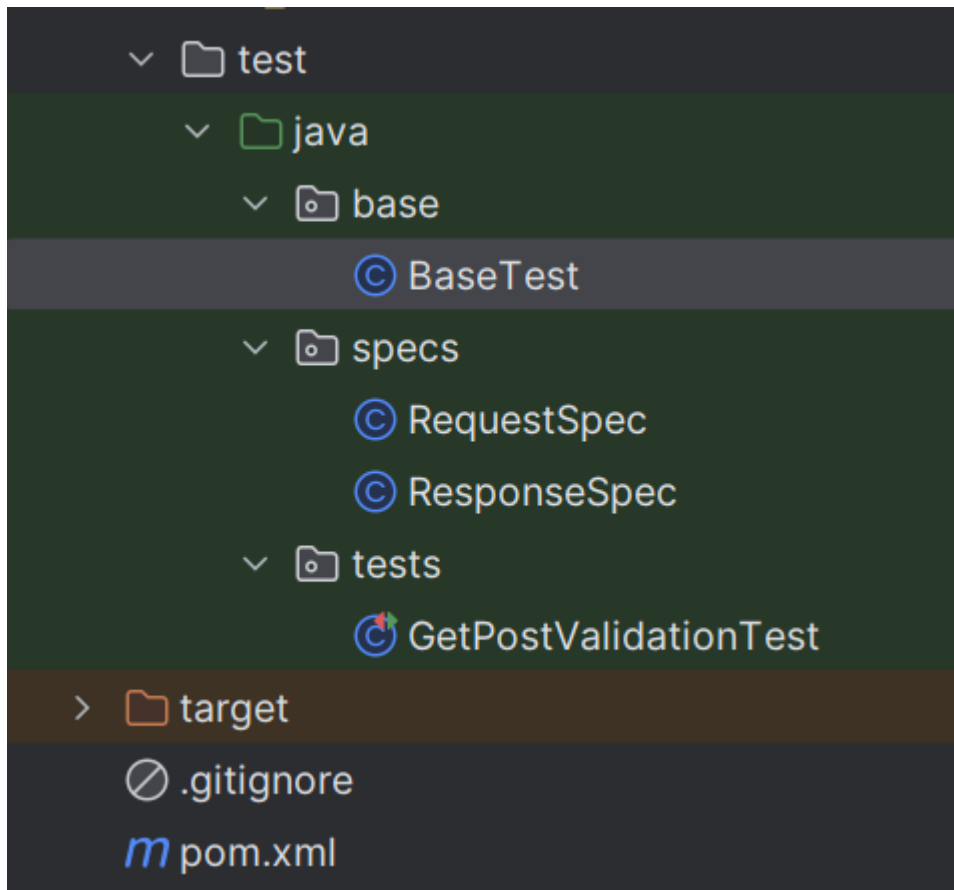
ApiTestingTest

src/test/java/com/example/api/base/BaseTest.java

src/test/java/com/example/api/factory/SpecFactory.java

src/test/java/com/example/api/tests/GetPostValidationTest.java

pom.xml



7. Test Scenario

Scenario Name

Validate Post Details API

8. Test Steps

Step 1

Set the Base URI:

<https://jsonplaceholder.typicode.com>

Step 2

Send GET request to:

/posts/1

Step 3

Capture API response

Step 4

Validate response status code

Expected:

200

Step 5

Validate response body fields

Checks performed:

- id should be equal to 1
- userId should be present
- title should not be empty
- body should not be empty

9. API Request

Method:

GET

Endpoint:

/posts/1

Full URL:

<https://jsonplaceholder.typicode.com/posts/1>

10. Sample API Response

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit suscipit recusandae consequuntur expedita et cum
reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet
architecto"
}
```

11. Validations Performed

Validation	Expected Result	Status
------------	-----------------	--------

Status Code	200	Passed
id	1	Passed
userId	Present	Passed
title	Not Empty	Passed
body	Not Empty	Passed

12. BaseTest.java

```
package com.example.api.base;

import io.restassured.RestAssured;
import org.junit.jupiter.api.BeforeAll;

public class BaseTest {

    @BeforeAll
    public static void setup() {

        RestAssured.baseURI =
            "https://jsonplaceholder.typicode.com";
    }
}
```

13. SpecFactory.java

```
package com.example.api.factory;
```

```
import io.restassured.builder.RequestSpecBuilder;
import io.restassured.builder.ResponseSpecBuilder;
import io.restassured.http.ContentType;
import io.restassured.specification.RequestSpecification;
import io.restassured.specification.ResponseSpecification;

public class SpecFactory {

    public static RequestSpecification getRequestSpec() {

        return new RequestSpecBuilder()
            .setContentType(ContentType.JSON)
            .build();
    }

    public static ResponseSpecification getResponseSpec() {

        return new ResponseSpecBuilder()
            .expectStatusCode(200)
            .build();
    }
}
```

14. GetPostValidationTest.java

```
package com.example.api.tests;

import com.example.api.base.BaseTest;
import com.example.api.factory.SpecFactory;
```

```
import io.restassured.response.Response;

import org.junit.jupiter.api.Test;


import static io.restassured.RestAssured.given;
import static org.hamcrest.Matchers.*;


public class GetPostValidationTest extends BaseTest {


    @Test
    public void validatePostDetailsTest() {


        Response response =

            given()

                .spec(SpecFactory.getRequestSpec())


            .when()

                .get("/posts/1")


            .then()

                .spec(SpecFactory.getResponseSpec())
                .body("id", equalTo(1))
                .body("userId", notNullValue())
                .body("title", not(emptyString()))
                .body("body", not(emptyString()))
                .extract()
                .response();
```

```
        response.prettyPrint();
    }
}
```

15. JSON Path Used

The following JSON Paths were validated:

JSON Path Validation

id	equalTo(1)
userId	notNullValue()
title	not(emptyString())
body	not(emptyString())

16. Assertions Used

Hamcrest assertions used:

- equalTo()
- notNullValue()
- emptyString()
- not()

These assertions ensure API response correctness.

```
given()
    .spec(RequestSpec.requestSpecification())

    .when()
    .get(s: "/posts/1")

    .then()
    .spec(ResponseSpec.responseSpecification())

    .body(s: "id", equalTo(operand: 1))

    .body(s: "userId", notNullValue())

    .body(s: "title", not(emptyOrNullString()))

    .body(s: "body", not(emptyOrNullString()));
```

17. Request Specification

Reusable Request Specification includes:

- Content Type
- Common request settings

Benefits:

- Reusability
- Cleaner code
- Easy maintenance

```

package specs;

import io.restassured.builder.RequestSpecBuilder;
import io.restassured.specification.RequestSpecification;

public class RequestSpec { 2 usages

    public static RequestSpecification requestSpecification() { 1 usage

        return new RequestSpecBuilder()
            .addHeader( headerName: "Content-Type", headerValue: "application/json")
            .build();
    }
}

```

18. Response Specification

Reusable Response Specification includes:

- Status Code Validation
- Common response validations

Benefits:

- Centralized validations
- Reduced duplicate code

```

package specs;

import io.restassured.builder.ResponseSpecBuilder;
import io.restassured.specification.ResponseSpecification;

public class ResponseSpec { no usages

    public static ResponseSpecification responseSpecification() { no usages

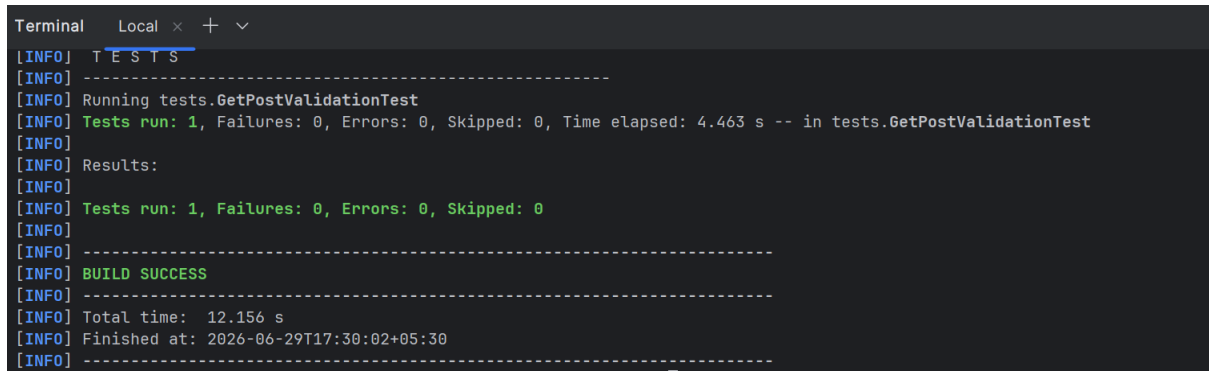
        return new ResponseSpecBuilder()
            .expectStatusCode( expectedStatusCode: 200)
            .build();
    }
}

```

19. Test Execution

Command used to execute tests:

mvn test

A terminal window titled 'Terminal' with tabs 'Local', 'x', '+', and 'v'. The output shows a successful Maven test run. It starts with '[INFO] T E S T S' and a separator line. Then '[INFO] Running tests.GetPostValidationTest'. The next line shows '[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.463 s -- in tests.GetPostValidationTest'. This is followed by '[INFO] Results:' and '[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0'. Another separator line is shown, followed by '[INFO] BUILD SUCCESS'. A final separator line is shown, then '[INFO] Total time: 12.156 s' and '[INFO] Finished at: 2026-06-29T17:30:02+05:30'. The last line is '[INFO]' followed by a separator line.

```
[INFO] T E S T S
[INFO] -----
[INFO] Running tests.GetPostValidationTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.463 s -- in tests.GetPostValidationTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 12.156 s
[INFO] Finished at: 2026-06-29T17:30:02+05:30
[INFO] -----
```

20. Expected Output

Test should execute successfully.

Console Output:

BUILD SUCCESS

API response should be printed successfully.

21. Key Learning Outcomes

This project helped in understanding:

- REST API Testing
 - Rest Assured Framework
 - GET API Automation
 - JSON Response Validation
 - JSON Path
 - Hamcrest Assertions
 - Maven Project Setup
 - Framework Design
 - Request & Response Specifications
-

22. Conclusion

The API automation framework was successfully implemented using Rest Assured.

The GET API endpoint:

/posts/1

was validated successfully with the following checks:

- Status code validation
- JSON response validation
- Mandatory field verification
- Assertions using Hamcrest

All test cases passed successfully.

Github Link: [Branches · ust-sdet-training/SDET](#)

Branchname:Karthikeyan_ApiSelenium_29jun